Intrainz Major Project

अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education

TOPIC: Online Retail Recommendation System

Name: S. Mythili

Domain: Data Science

InTrAinZ

IGNITING A BRIGHTER FUTURE

Introduction

In today's competitive online retail environment, providing personalized recommendations to customers has become crucial for enhancing user experience and increasing sales. Recommendation systems are powerful tools that help businesses achieve these goals by analyzing user behavior and suggesting products that users are likely to be interested in. This project focuses on developing a recommendation system for an online retail dataset using user-based collaborative filtering.

The dataset used in this project contains transactional data from an online retail store, including information about purchases made by customers over a period of time. Each transaction records details such as the customer ID, product ID, quantity purchased, and the date of purchase. By leveraging this data, we aim to build a system that can predict and recommend products to customers based on their purchase history and the purchasing patterns of similar users.

The primary objective of this project is to create a simple yet effective recommendation system that can be easily understood and implemented. We have chosen user-based collaborative filtering as our approach, which involves identifying similarities between users to make recommendations. This method is intuitive and widely used in recommendation systems due to its simplicity and effectiveness.

1. Load Data

- **Purpose:** Load the data from the provided Excel file.

```
In [ ]:
import pandas as pd

# Load the first 5000 rows from the dataset
file_path = "OnlineRetail (1).xlsx'
data = pd.read_excel(file_path, nrows=5000)
print(data.head())

```

- **Explanation:**
  - import pandas as pd imports the pandas library, which is used for data manipulation.
  - pd.read_excel(file_path, nrows=5000) reads the first 5000 rows from the Excel file located at file_path.
  - print(data.head()) prints the first few rows of the loaded data to give an idea of its structure.

2. Data Preprocessing

- **Purpose:** Clean and prepare the data for analysis.

```
# Drop rows with any missing values
data.dropna(inplace=True)

# Filter relevant columns
data = data[['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'UnitPrice', 'CustomerID', 'Country']]

# Convert necessary columns to appropriate data types
data['InvoiceDate'] = pd.to_datetime(data['InvoiceDate'])
data['CustomerID'] = data['CustomerID'].astype('int64')

```

- **Explanation:**
  - data.dropna(inplace=True) removes any rows that have missing values.
  - data = data[['...']] selects the relevant columns that we will use for analysis.

- data['InvoiceDate'] = pd.to_datetime(data['InvoiceDate']) converts the InvoiceDate column to a date-time format.
- data['CustomerID'] = data['CustomerID'].astype('int64') converts the CustomerID column to integer type.

3. Exploratory Data Analysis

- **Purpose:** Understand the data better.

In [ ]:
```
# Basic statistics
print(data.describe())

# Unique customers and products
print(f"Unique customers: {data['CustomerID'].nunique()}")
print(f"Unique products: {data['StockCode'].nunique()}")

```

- **Explanation:**
  - data.describe() prints basic statistics about the numerical columns in the data.
  - data['CustomerID'].nunique() counts the number of unique customers.
  - data['StockCode'].nunique() counts the number of unique products.

4. Model Selection

We'll use a simple User-Based Collaborative Filtering approach.

In [ ]:
```
import pandas as pd
from surprise import Dataset, Reader, KNNBasic, accuracy
from surprise.model_selection import train_test_split

# Load the first 5000 rows from the dataset
file_path = 'path_to_your_file/OnlineRetail (1).xlsx'
data = pd.read_excel(file_path, nrows=5000)

# Drop rows with any missing values
data.dropna(inplace=True)

# Filter relevant columns
data = data[['CustomerID', 'StockCode', 'Quantity']]


```

```
In [ ]:
reader = Reader(rating_scale=(1, 5))
data_for_surprise = Dataset.load_from_df(data[['CustomerID', 'StockCode', 'Quantity']], reader)

# Split the data into training and test sets
trainset, testset = train_test_split(data_for_surprise, test_size=0.2)

# Initialize the KNNBasic algorithm for User-Based Collaborative Filtering
algo = KNNBasic()

# Train the model on the training set
algo.fit(trainset)

# Generate predictions for the test set
predictions = algo.test(testset)

# Evaluate the model's accuracy using RMSE
print(accuracy.rmse(predictions))
```

5. Model Training

- **Purpose:** Train a recommendation model.

```
In [ ]:
from sklearn.model_selection import train_test_split
from surprise import Dataset, Reader, KNNBasic
from surprise.model_selection import train_test_split as surprise_train_test_split
from surprise import accuracy

# Prepare data for surprise library
reader = Reader(rating_scale=(1, 5))
data_for_surprise = Dataset.load_from_df(data[['CustomerID', 'StockCode', 'Quantity']], reader)

# Split data into training and test sets
trainset, testset = surprise_train_test_split(data_for_surprise, test_size=0.2)

# Use KNNBasic for User-Based Collaborative Filtering
algo = KNNBasic()
algo.fit(trainset)
```

- **Explanation:**
  - from sklearn.model_selection import train_test_split imports the function to split data into training and testing sets.
  - from surprise import ... imports necessary classes from the Surprise library, which is used for building recommendation systems.
  - reader = Reader(rating_scale=(1, 5)) sets the rating scale for the data.

- Dataset.load_from_df(data[['CustomerID', 'StockCode', 'Quantity']], reader) loads the data into a format suitable for Surprise.
- surprise_train_test_split(data_for_surprise, test_size=0.2) splits the data into 80% training and 20% testing sets.
- algo = KNNBasic() initializes a basic k-nearest neighbors algorithm.
- algo.fit(trainset) trains the model using the training data.

6. Model Evaluation

- **Purpose:** Evaluate how well the model performs.

In [ ]:
```
# Evaluate the model
predictions = algo.test(testset)
accuracy.rmse(predictions)
```

- **Explanation:**
  - predictions = algo.test(testset) generates predictions for the test data.
  - accuracy.rmse(predictions) calculates the Root Mean Square Error (RMSE) to measure the accuracy of the predictions.

7. Generate Recommendations

- **Purpose:** Generate top product recommendations for each user.

In [ ]:

```
from collections import defaultdict

def get_top_n(predictions, n=10):
    top_n = defaultdict(list)
    for uid, iid, true_r, est, _ in predictions:
        top_n[uid].append((iid, est))
    for uid, user_ratings in top_n.items():
        user_ratings.sort(key=lambda x: x[1], reverse=True)
        top_n[uid] = user_ratings[:n]
    return top_n

# Get top 10 recommendations for each user
top_n_recommendations = get_top_n(predictions, n=10)
print(top_n_recommendations)
```

- **Explanation:**
  - from collections import defaultdict imports a dictionary subclass for storing lists.
  - def get_top_n(predictions, n=10) defines a function to get the top n recommendations.
  - Inside the function, it loops through each prediction and appends the product and estimated rating to the user's list.
  - user_ratings.sort(...) sorts the user's product list by the estimated rating in descending order.
  - top_n[uid] = user_ratings[:n] keeps only the top n products for each user.
  - get_top_n(predictions, n=10) generates the top 10 recommendations for each user and stores them in top_n_recommendations.

Significance

Implementing an effective recommendation system has several significant benefits for online retail businesses:

- Enhanced User Experience: Personalized recommendations can help users discover products they might be interested in, improving their overall shopping experience.
- Increased Sales: By suggesting relevant products, recommendation systems can boost sales and increase the average order value.
- Customer Retention: Personalized recommendations can enhance customer satisfaction and loyalty, encouraging repeat purchases.
- Data Utilization: Leveraging transactional data for recommendations allows businesses to make data-driven decisions and optimize their marketing strategies.

Scope

This project provides a foundation for understanding and implementing a basic recommendation system using user-based collaborative filtering. While the scope of this project is limited to a simple model, it serves as a stepping stone for further enhancements and more advanced recommendation algorithms. The techniques and methodologies discussed in this project can be extended and refined to develop more sophisticated systems capable of handling larger datasets and providing more accurate recommendations**.**

Conclusion:

In this project, we developed a simple and effective recommendation system for an online retail dataset using user-based collaborative filtering. Here's a summary of the steps we followed and the results achieved:

1. **Data Loading and Preprocessing:**
    - We successfully loaded the first 5000 rows from the provided online retail dataset.
    - Data cleaning involved removing rows with missing values and selecting relevant columns for our analysis.
    - We converted necessary columns to appropriate data types to ensure consistency.
2. **Exploratory Data Analysis:**

    - We explored basic statistics of the dataset and identified the number of unique customers and products.
    - This helped us understand the scale and diversity of the data.
3. **Model Selection:**
    - We selected user-based collaborative filtering for our recommendation system. This method uses the similarity between users to make recommendations.
    - We utilized the KNNBasic algorithm from the Surprise library, which is a k-nearest neighbors algorithm suitable for collaborative filtering tasks.
4. **Model Training and Evaluation:**
    - The dataset was split into training (80%) and test (20%) sets to ensure a fair evaluation of the model.
    - The KNNBasic algorithm was trained on the training set and evaluated on the test set.
    - We used Root Mean Square Error (RMSE) as our evaluation metric, which measures the accuracy of the predicted ratings compared to the actual ratings.
5. **Generating Recommendations:**
    - After training the model, we generated product recommendations for users based on their similarity to other users.
    - The top recommendations were extracted and can be used to personalize the shopping experience for users.

Key Findings:

- **Model Performance:** The model's performance, measured by RMSE, indicates how closely the predicted ratings matched the actual ratings. Lower RMSE values suggest better accuracy.
- **User Personalization:** User-based collaborative filtering proved effective in generating personalized recommendations by leveraging the similarities between users' past behaviors.
- **Scalability:** The approach can be scaled to larger datasets, making it suitable for real-world applications in online retail platforms.

Future Work:

- **Enhanced Data Preprocessing:** Further data cleaning and feature engineering, such as handling outliers and creating new features, could improve model performance.
- **Advanced Models:** Exploring more advanced recommendation algorithms, such as matrix factorization or hybrid models, could provide better recommendations.
- **Real-Time Recommendations:** Implementing real-time recommendation capabilities to provide instant suggestions as users interact with the platform.

This project demonstrated a practical approach to building a recommendation system using user-based collaborative filtering. By following a structured methodology, we successfully developed a model that can provide personalized product recommendations to users. This system has the potential to enhance user experience and increase sales by suggesting relevant products based on users' preferences and behaviors.